

# Mukaputa

Project Technical Documentation  
Frontend | Backend | Database | Real-Time

Version 1.0 | September 2026 | Confidential

| Layer        | Technology              | Purpose                               |
|--------------|-------------------------|---------------------------------------|
| Frontend     | HTML / CSS / JS         | User Interface runs in the browser    |
| Backend      | Python / Flask          | API Server handles all business logic |
| Real-Time    | Socket.IO + Eventlet    | Live calls, notifications, chat       |
| Database     | PostgreSQL via Supabase | Stores all user data in the cloud     |
| File Storage | Supabase Storage        | Uploaded photos and videos            |

---

# 1. Project Overview

**Mukaputa** is a full-stack social media platform built from scratch with Python on the server and plain JavaScript on the client. Users can post, comment, react, upload reels, live-stream, chat, make video calls, apply to jobs, and more.

## Core Features

- Posts with text, photos, and emoji reactions (Like, Love, Haha, Wow, Sad, Angry)
- Comments, nested replies, and the ability to edit or delete your own comments
- Reels (short videos) with likes, comments, and a full-screen swipe player
- Stories that disappear after 24 hours
- Messenger with real-time chat, image sharing, and voice notes
- Voice and Video calls using WebRTC (direct browser-to-browser, no server relay)
- Live streaming: broadcast video to all followers in real time
- Friends, Follow, and Block system
- Real-time push notifications
- Job listings and one-click job applications
- Dark / Light mode toggle
- Google OAuth login alongside email and password authentication
- Deactivate account (recoverable) or permanently delete account and all data

---

## 2. Project File Structure

```
mukaputa/
|-- run.py           Entry point. Run this to start the server.
|-- start.bat        One-click Windows launcher (install + start + open browser)
|-- requirements.txt List of Python packages needed
|-- .env             Secret keys and database URL (never share this file!)
|-- mukaputa.db      Local SQLite fallback (used when no Supabase URL is set)
|
|-- backend/         All Python / Flask server code
|   |-- app.py        App factory - creates and wires up the Flask application
|   |-- config.py     Settings loaded from .env (DB URL, secret key, limits)
|   |-- extensions.py Shared objects: db, login_manager, socketio
|   |-- models.py     Database table definitions (one class = one table)
|   |-- sockets.py    Real-time WebSocket event handlers (calls, live stream)
|   |-- seed.py       Fills database with demo data on first run
|   |-- utils.py      ok() and err() helper functions for JSON responses
|   |-- routes/
|       |-- auth.py   /api/auth/ Login, register, logout, delete account
|       |-- bootstrap.py /api/bootstrap Loads all data at once on app start
|       |-- posts.py  /api/posts/ Create, edit, delete posts and comments
|       |-- social.py /api/social/ Friends, follow, block, search users
|       |-- content.py /api/content/ Reels, stories, reactions, saved posts
|       |-- messaging.py /api/messages/ Send and receive chat messages
|       |-- notifications.py /api/notifications/ Get and mark notifications
|       |-- jobs.py   /api/jobs/ Job listings and applications
|       |-- uploads.py /api/upload/ Photo and video file uploads
|
|-- frontend/        All HTML / CSS / JavaScript code
|   |-- index.html   Main dashboard (entire app in one HTML file)
|   |-- login.html   Login and register page
|   |-- landing_page.html Public homepage for visitors
|   |-- css/
|       |-- variables.css Theme colours, fonts, spacing (dark/light mode)
|       |-- layout.css   Page grid, navbar, sidebar, mobile bottom bar
|       |-- feed.css     Post cards, stories tray, reels, comments
|       |-- components.css Buttons, modals, badges, cards
|       |-- realtime.css Call overlay UI, live stream UI
|   |-- js/
|       |-- store.js    Data layer and API client (the app's memory)
|       |-- components.js HTML builder functions (renderPost, renderReel, etc.)
|       |-- feed.js     Home feed renderer
|       |-- app.js      Main app controller (handles all user interactions)
|       |-- calls.js    WebRTC voice and video call management
|       |-- login.js    Login / register page logic
|       |-- boot.js     App startup: check session, load data, show app
|
|-- uploads/         Uploaded media files saved on the server
```

---

## 3. Backend (Python / Flask)

### 3.1 run.py - Entry Point

This is the file you run to start the server. It imports `create_app()` from `backend/app.py` and the `socketio` instance from `extensions.py`, then calls `socketio.run()` which starts the Eventlet web server on port 5000.

### 3.2 backend/app.py - App Factory

The `create_app()` function does the following in order:

- Loads settings from the Config class (which reads `.env`)
- Initialises `db`, `login_manager`, and `socketio` with the app
- Registers all route Blueprints (`auth`, `posts`, `social`, `content`, etc.)
- Sets up routes to serve the frontend HTML, CSS, and JS files
- Creates all database tables with `db.create_all()`
- Runs `seed.py` to fill the database with demo data if it is empty

### 3.3 backend/config.py - Settings

| Setting                    | Value Source                            | Description                                          |
|----------------------------|-----------------------------------------|------------------------------------------------------|
| SECRET_KEY                 | <code>.env</code> MUKAPUTA_SECRET_KEY   | Encrypts browser session cookies                     |
| SQLALCHEMY_DATABASE_URI    | <code>.env</code> MUKAPUTA_DATABASE_URL | PostgreSQL connection string                         |
| MAX_CONTENT_LENGTH         | 25 MB hard-coded                        | Maximum allowed upload file size                     |
| PERMANENT_SESSION_LIFETIME | 30 days                                 | How long a login session stays valid                 |
| SESSION_COOKIE_HTTPONLY    | <code>True</code>                       | Blocks JavaScript from reading the cookie (security) |

### 3.4 backend/extensions.py

Creates three shared singleton objects used across the whole app:

- **db** - SQLAlchemy instance for all database read/write operations
- **login\_manager** - Flask-Login for session and authentication management
- **socketio** - Flask-SocketIO for real-time WebSocket connections (`calls`, `live`)

### 3.5 backend/models.py - Database Tables

Each Python class in this file maps directly to one table in PostgreSQL. SQLAlchemy automatically converts Python operations into SQL queries.

| Class (= Table) | Key Columns                                         | Purpose                         |
|-----------------|-----------------------------------------------------|---------------------------------|
| User            | id, name, username, email, password_hash, is_active | All user accounts               |
| Post            | id, author_id, text, media, feeling, created_at     | Feed posts                      |
| Comment         | id, post_id, author_id, text                        | Comments on posts               |
| CommentReply    | id, comment_id, author_id, text                     | Nested replies to comments      |
| PostReaction    | id, post_id, user_id, reaction_type                 | Likes, loves, hahas, etc.       |
| Reel            | id, author_id, video_url, caption, created_at       | Short video reels               |
| ReelLike        | user_id, reel_id                                    | Who liked which reel            |
| ReelComment     | id, reel_id, author_id, text                        | Comments on reels               |
| Story           | id, author_id, media_url, expires_at                | 24-hour stories                 |
| FriendLink      | user_id, friend_id, status                          | Friend requests and connections |
| Follow          | follower_id, followee_id                            | Follow relationships            |
| BlockedUser     | user_id, blocked_id                                 | Blocked users list              |
| Conversation    | id, created_at                                      | A chat room between two users   |
| Message         | id, conversation_id, sender_id, content, type       | Chat messages                   |
| Notification    | id, user_id, actor_id, type, read                   | All in-app notifications        |
| SavedItem       | user_id, post_id                                    | Posts saved by a user           |
| JobApplication  | user_id, job_id, applied_at                         | Job applications submitted      |

### 3.6 backend/routes/ - API Endpoints

| File         | URL Prefix     | Key Endpoints                                                                |
|--------------|----------------|------------------------------------------------------------------------------|
| auth.py      | /api/auth/     | POST /login, POST /register, POST /logout, DELETE /account, POST /deactivate |
| bootstrap.py | /api/bootstrap | GET - returns ALL data in one shot when app loads                            |
| posts.py     | /api/posts/    | GET, POST (create), PUT (edit), DELETE, comments, reactions                  |
| social.py    | /api/social/   | Add/remove friends, follow, block, search users                              |
| content.py   | /api/content/  | Reels CRUD, stories, saved posts                                             |
| messaging.py | /api/messages/ | List conversations, send and fetch messages                                  |

| File             | URL Prefix          | Key Endpoints                                     |
|------------------|---------------------|---------------------------------------------------|
| notifications.py | /api/notifications/ | Get notifications, mark as read                   |
| jobs.py          | /api/jobs/          | Browse job listings, submit application           |
| uploads.py       | /api/upload/        | POST file, returns public URL for frontend to use |

### 3.7 backend/sockets.py - Real-Time Events

Uses Socket.IO over WebSockets. On connect, each user joins a private room named after their user ID. Events are routed to specific users by room name.

| Event               | Direction        | What it does                                        |
|---------------------|------------------|-----------------------------------------------------|
| connect             | Client to Server | User joins their personal private room              |
| call:invite         | Client to Server | Initiate a voice or video call to another user      |
| call:accept         | Client to Server | Accept an incoming call                             |
| call:signal         | Both directions  | Exchange WebRTC SDP offer/answer and ICE candidates |
| call:decline        | Client to Server | Decline or end an active call                       |
| live:start          | Client to Server | Begin a live video broadcast to all followers       |
| live:end            | Client to Server | End the live broadcast                              |
| live:join_request   | Client to Server | A viewer requests to join a live stream             |
| live:offer / answer | Both             | WebRTC negotiation for each live stream viewer      |

---

## 4. Frontend (HTML / CSS / JavaScript)

The frontend is a **Single Page Application (SPA)**. There is only one HTML file (index.html). JavaScript dynamically replaces the page content without any full page reload. All data comes from the backend API as JSON.

### 4.1 Script Load Order

| Order | File          | Why this order                                                          |
|-------|---------------|-------------------------------------------------------------------------|
| 1st   | store.js      | Must load first. All other scripts depend on the 'store' global object. |
| 2nd   | components.js | Uses store to get data and turns it into HTML strings.                  |
| 3rd   | feed.js       | Uses components.js to render the home feed posts.                       |
| 4th   | app.js        | Uses all of the above. Defines the 'app' controller class.              |
| 5th   | calls.js      | Uses app and store for WebRTC voice and video call management.          |
| 6th   | boot.js       | Runs LAST. Boots the entire app after all other scripts are ready.      |

### 4.2 store.js - Data Layer

The most important JS file. Acts as the app's memory and its API client.

- **store.db** - holds all data in memory after boot (users, posts, reels, chats)
- **store.\_api(method, url, body)** - wrapper around fetch() for all backend calls
- **store.boot()** - checks login session, loads all data from /api/bootstrap
- **store.getCurrentUser()** - returns the logged-in user object
- **store.getPosts()** - returns all posts in memory
- **store.getUser(id)** - looks up a user by their ID
- **store.deletePost(id)** - removes from memory AND calls DELETE API
- **store.sendMessage()** and **refreshConversation()** - real-time chat helpers

### 4.3 components.js - HTML Builders

Pure functions that take data objects and return HTML strings.

| Function                     | What it builds                                                        |
|------------------------------|-----------------------------------------------------------------------|
| renderPost(post)             | Full post card with header, media, reaction bar, comments, 3-dot menu |
| renderReel(reel)             | Full-screen reel with video element, like/comment/share buttons       |
| renderProfileView(user, tab) | Full profile page: cover photo, avatar, stats, nav tabs, content      |

| Function                            | What it builds                                                 |
|-------------------------------------|----------------------------------------------------------------|
| <code>renderComment(comment)</code> | Single comment bubble with replies, edit and delete buttons    |
| <code>renderStory(story)</code>     | Story circle card in the stories tray at the top of the feed   |
| <code>renderJobCard(job)</code>     | A job listing card with company, title, location, apply button |
| <code>renderFriendCard(user)</code> | A friend suggestion card with add/ignore buttons               |

## 4.4 app.js - Main Controller

The App class - the brain of the frontend. Every button click calls a method here.

- **switchView(name, params)** - navigate between Home, Profile, Reels, Messages, Jobs
- **renderProfileView(userId)** - renders a user's full profile page
- **createPost()** - reads the post compose box and sends data to the API
- **deletePost(postId)** - confirms, calls API, fades the card out of the DOM
- **togglePostMenu(postId)** - opens or closes the 3-dot options dropdown
- **reactPost(postId, type)** - sends a reaction (Like, Love, Haha, etc.)
- **openReelComments(reelId)** - slides open the comment panel on a reel
- **confirmDeactivateAccount()** - deactivates account and redirects to login
- **confirmDeleteAccount()** - permanently deletes all data and account
- **startPolling()** - polls every 4 seconds for new messages and notifications
- **showToast(msg, type)** - shows a pop-up notification toast at the bottom

## 4.5 calls.js - WebRTC Voice and Video Calls

Manages peer-to-peer calls. Socket.IO handles only the signaling (exchanging connection info). The actual audio and video stream goes directly between the two browsers without passing through the server.

- Uses `RTCPeerConnection` for direct browser-to-browser media streaming
- Uses `navigator.mediaDevices.getUserMedia()` to request camera and microphone
- Requires HTTPS or localhost - plain HTTP on a LAN IP (e.g. 192.168.x.x) blocks camera on mobile
- Call flow: invite -> accept -> SDP offer/answer -> ICE candidates -> connected
- Shows a clear error message if the browser blocks camera access

## 4.6 boot.js - App Startup Sequence

- Calls `store.boot()` which hits `GET /api/auth/me` to check if user is logged in
- If not logged in, redirects immediately to `login.html`
- If logged in, calls `GET /api/bootstrap` to load all data in a single request
- Calls `app.init()` to render the first view of the dashboard
- Calls `app.startPolling()` to begin background refresh every 4 seconds

- Calls `Calls.init()` to connect Socket.IO for real-time features
- Dismisses the splash screen with a smooth 0.5s fade-out animation

---

## 5. Database (PostgreSQL on Supabase)

Mukaputa uses **PostgreSQL** hosted on **Supabase** (free cloud tier). Locally it falls back to a SQLite file (mukaputa.db) if no DATABASE\_URL is set. The ORM (SQLAlchemy) handles all SQL - you never write raw SQL queries.

### 5.1 Connection String (.env)

```
MUKAPUTA_DATABASE_URL=postgresql://postgres.xxxx:Password@host:5432/postgres
```

SQLAlchemy reads this string and creates a connection pool automatically.  
All database queries in models.py and routes/ go through this pool.

### 5.2 Table Relationships

| Relationship               | Type         | How it works in code                            |
|----------------------------|--------------|-------------------------------------------------|
| User -> Post               | One to Many  | Post.author_id is a foreign key to User.id      |
| Post -> Comment            | One to Many  | Comment.post_id is a foreign key to Post.id     |
| Comment -> CommentReply    | One to Many  | CommentReply.comment_id FK to Comment.id        |
| Post -> PostReaction       | One to Many  | PostReaction holds user_id + post_id + type     |
| User <-> User (FriendLink) | Many to Many | FriendLink has user_id, friend_id, and status   |
| User <-> User (Follow)     | Many to Many | Follow table stores follower_id and followee_id |
| Conversation -> Message    | One to Many  | Message.conversation_id FK to Conversation.id   |
| User <-> Conversation      | Many to Many | ConversationParticipant join table              |

---

## 6. End-to-End Data Flow Examples

### 6.1 Creating a Post

1. User types in the Create Post box and clicks Post
2. `app.js createPost()` reads the text and any attached media
3. `store.js _api('POST', '/api/posts', { text, media })`  
sends the data to the backend as JSON
4. `backend/routes/posts.py create_post()`  
creates a `Post()` model object  
`db.session.add(post)` and `db.session.commit()`  
returns `{ post: {...} }` as JSON
5. `store.js` adds the new post to `store.db.posts` in memory
6. `feed.js feed.render()` calls `renderPost()` for every post
7. `components.js renderPost()` returns the HTML string for the card
8. The DOM updates and the user sees their new post instantly

### 6.2 Deleting a Post from Profile

1. User clicks the 3-dot button on a post in their profile
  - The dropdown was invisible because `.post` had `overflow:hidden` in CSS
  - Fixed by changing `.post` to `overflow:visible`
2. `togglePostMenu(postId)` shows the dropdown menu
3. User clicks Delete Post
4. `app.js deletePost(postId)` shows `confirm()` dialog
5. If confirmed:  
`store.deletePost(postId)` calls `DELETE /api/posts/{id}`  
`backend/routes/posts.py` deletes the Post from the database
6. The post card fades to opacity 0 in 200ms then is removed from DOM
7. After 250ms, `renderProfileView()` re-renders the profile  
so the post count in the stats bar updates too

### 6.3 Making a Video Call

1. User A clicks the video call button on User B's profile
2. `calls.js startCall(userId, 'video')`  
`getUserMedia()` requests camera and microphone access  
`socket.emit('call:invite', { to: userBId, type: 'video' })`
3. Server `sockets.py handle_call_invite()`  
`emit('call:invite', data, room=userBId)`  
-- sends directly to User B's browser
4. User B sees the incoming call popup with Accept and Decline
5. User B clicks Accept  
`getUserMedia()` gets their camera and mic  
`socket.emit('call:accept', { to: userAId })`
6. Server forwards 'call:accept' to User A
7. User A creates `RTCPeerConnection`  
creates an SDP offer and sends it via `socket.emit('call:signal')`
8. User B receives the offer, creates an SDP answer, sends it back
9. Both browsers exchange ICE candidates via 'call:signal'

10. WebRTC peer connection established  
Audio and video now flow directly between the two browsers

## 6.4 Login with a Deactivated Account

1. User enters their email and password on login.html
2. login.js sends POST /api/auth/login with { identifier, password }
3. backend/routes/auth.py login()  
finds user by email or username in the database  
verifies the password hash  
calls login\_user(user, force=True)  
-- force=True bypasses Flask-Login's is\_active check  
-- without force=True, a deactivated account would be blocked here  
if user.is\_active is False:  
    user.is\_active = True      (reactivates the account)  
    db.session.commit()  
returns { user: { id, name, username, ... } }
4. login.js redirects to index.html
5. boot.js loads all data and starts the app normally

---

# 7. Full Technology Stack

| Category       | Technology         | Version   | Purpose                                                 |
|----------------|--------------------|-----------|---------------------------------------------------------|
| Language       | Python             | 3.11      | Backend server language                                 |
| Web Framework  | Flask              | 3.0       | Creates APIs, serves frontend files, handles requests   |
| Real-Time      | Flask-SocketIO     | 5.6       | WebSocket support for calls, live stream, notifications |
| Async Server   | Eventlet           | 0.41      | Production WebSocket server that replaces Werkzeug      |
| ORM            | SQLAlchemy         | 2.0       | Python-to-SQL bridge for all database operations        |
| Authentication | Flask-Login        | 0.6       | Session management, login required decorator            |
| Database       | PostgreSQL         | 15+       | Production relational database                          |
| DB Hosting     | Supabase           | Free tier | Managed cloud PostgreSQL with storage                   |
| Frontend       | Vanilla JavaScript | ES2022    | No framework - pure JS for full control                 |
| Styling        | Custom CSS3        | CSS3      | No Bootstrap - hand-crafted styles                      |
| Video Calls    | WebRTC             | —         | Browser-native peer-to-peer audio and video             |
| OAuth          | Google OAuth 2.0   | —         | Sign in with Google                                     |
| PDF            | ReportLab          | 5.0       | Generated this documentation file                       |